UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

Raport
Proiect SLCheck

Apostu Cleopatra
Pricop Ania
Grupa 144
05.01.2026

1. Prezentarea cerinței

SLCheck (1–2 studenți) – Scrieți un program (script) shell care primește ca parametru un director (eventual chiar rădăcina sistemului de fișiere) și verifică dacă există broken links, link-uri simbolice care referă fișiere inexistente. Programul funcționează recursiv, verificând întreaga ierarhie de fișiere și directoare accesibilă pornind din directorul furnizat ca parametru al programului. Atunci când întlnește un link simbolic către un director, comportamentul implicit al programului este să nu dereferențieze link-ul, adică să nu pornească recursiv analiza directorului referențiat. Acest comportament se poate schimba dacă programul este apelat cu flag-ul –follow-symlinks, caz în care analiza recursivă continuă cu directorul referit în link-ul simbolic.

Indicație: Cel mai simplu mod de a genera broken links este să folosiți un sistem de fișiere tip tmpfs pe care să copiați o structură de directoare, de pildă /etc. La dezinstalarea sistemului de fișiere (operație de umount), datele dispar automat și eventualele link-uri simbolice create către RAM disc devin broken. N.B. Parcurgerea în adâncime a ierarhiei de directoare trebuie implementată direct în program, nefiind permisă folosirea unor comenzi shell care ajută în acest task, cum ar fi comanda find.

O legătură simbolică (symlink) este un tip de fișier care face referire către un alt fișier sau un director prin stocarea unei căi către aceasta, oferind o cale alternativă de acces fără a copia conținutul țintei. O legătură simbolică se poate defecta dacă ținta sa este mutată sau ștearsă.

Un inode (index node) este o structură de date esențială în sistemele de fișiere Unix care stochează informații despre un fișier sau un director. Aceste informații cuprind proprietarul, permisiunile, dimensiunea și timestamp-urile (creare, modificare, acces). Nu conține informații despre numele fișierului sau conținut. Fiecare fișier și director are un inode unic pe sistemul de fișiere pe care se află.

Device ID-ul identifică unitatea de stocare (partiția).

2. Descrierea programului

a) Verificarea apelului

```
FOLLOW_SYMLINKS=false
TARGET_DIR=""

for arg in "$@"; do
```

```
    if [ "$arg" = "--follow-symlinks" ]; then
        FOLLOW_SYMLINKS=true
    elif [ -z "$TARGET_DIR" ]; then
        TARGET_DIR="$arg"
    fi
done

if [ -z "$TARGET_DIR" ]; then
    echo "Utilizare: $0 [--follow-symlinks] <director>"
    exit 1
fi

if [ ! -d "$TARGET_DIR" ]; then
    echo "Eroare: $TARGET_DIR nu este un director valid"
    exit 1
fi
declare -A VISITED_INODES
```

Mai sus avem prima parte a proiectului, cea în care verificăm argumentele primite și stabilim modul de funcționare.

În bucla for parcurgem toate argumentele transmise la rularea scriptului. Variabila arg va conține la fiecare iterație un argument diferit.

Dacă unul dintre argumente este ”--follow-symlinks”, atunci variabila FOLLOW_SYMLINKS este setată la true. Astfel este stabilit felul în care va avea loc parcurgelea directoarelor. Dacă variabila are valoarea true, programul va intra recursiv în directoarele referite de symlink-uri.

Cu testul -z este verificată lungimea valorii variabilei TARGET_DIR. Astfel, în cazul în care argumentul nu este --follow-symlinks și nu a fost găsit încă un director de pornire, argumentul curent devine TARGET_DIR.

Dacă după verificarea tuturor argumentelor încă nu a fost găsit un director de pornire, utilizatorul este anunțat cu privire la modalitatea corectă de apel și cu exit 1 este încheiată rularea.

Ulterior, este verificat dacă variabila TARGET_DIR conține adresa unui director existent, iar în caz contrar utilizatorul este anunțat și rularea se oprește.

Este decclarat un vector asociatic pentru verificarea perechilor device ID - inode.

b) Funcția process_directory()

```
process_directory() {
  local current_dir="$1"
```

```
  if [ ! -r "$current_dir" ] || [ ! -x "$current_dir" ]; then
      echo "Avertisment: Nu pot accesa directorul $current_dir"
      return
   fi

 local dir_id=$(stat -c "%d:%i" "$current_dir")
if [ -n "${VISITED_INODES[$dir_id]}" ]; then
echo " Ciclu detectat sau director deja vizitat: $current_dir"
return
fi
VISITED_INODES[$dir_id]=1



   for entry in "$current_dir"/*; do

      if [ -L "$entry" ]; then


         if [ ! -e "$entry" ]; then
            echo "[BROKEN LINK] $entry"
            continue
         fi

         if [ -d "$entry" ] && [ "$FOLLOW_SYMLINKS" = true ]; then
            process_directory "$entry"
         fi

      elif [ -d "$entry" ]; then
         process_directory "$entry"
      fi

   done
}
```

Funcția process_directory() implementează parcurgerea ierarhiei de directoare. Identifică link-urile simbolice și le detectează pe cele stricate.

Primește ca parametru calea directorului curent pe care o salvează în variabila locală current_dir. Variabila este locală pentru a nu risca suprascrierea variabilelor globale.

Cu -r verificam drepul de citire și cu -x dreptul de execuție pentru a lista fișierele și accesa inode-urile. Dacă nu avem cel puțin una dintre cele două permisiuni active, este afișat un avertisment. Este folosit return și nu exit în acest caz pentru că este oprită doar execuția pentr directorul curent, nu a întregului arbore de analizat.

Verificăm dacă directorul a mai fost vizitat și dacă „vizita” a provenit din aceeași sursă. Salvam atât device ID-ul cât si inode-ul pentru a fi sigură unicitatea codului pentru recunoastere. (spre exemplu, dacă folosim un laptop si un stick usb, două directoare diferite pot avea același inode. Inode-ul este unic doar în interiorul aceleiași partiții (sistem de fișiere).

În continuare, este folosită o buclă for pentru a trece prin toate elementele(fișiere, directoare, symlink-uri) din directorul curent. Variablia entry ia pe rând toate valorile. Este folosit * pentru a nu omite nicio intrare.

Folosind -L este testat dacă intrarea curentă este un link simbolic, indiferent dacă este valid sau nu, apoi cu -e verificăm existența țintei linkului găsit anterior. În cazul în care este broken, utilizatorul este anunțat și rularea este continuată pentru următoarea intrare.

Dacă un symlink duce către un director, este verificată valoarea variabilei FOLLOW_SYMLINKS pentri a stabili dacă funcția va fi apelată și pentru acel director sau nu.

Dacă directorul este deja dat prin apelarea funcției, acesta este parcurs.

c) finalul

```
echo "Analizez directorul: $TARGET_DIR"
process_directory "$TARGET_DIR"
```

Utilizatorul este informat cu privire la directorul procesat și funcția este apelată.

3. Dificultăți întâmpinate

a) Am întâmpinat problema unor bucle infinite în cazul în care aveam două directoare care aveau linkuri simbolice care duceau unul la altul. Această problemă a fost rezolvată cu ajutorul unui vector care reține inode-urile care au fost vizitate(împreună cu device ID-ul). Cum fiecare fișier are un inode unic, o vizitare de două ori duce la anunțarea buclei. Device-ID-ul asigură un cod unic pentru recunoaștere.
b) Încercarea de a accesa directoare fără a avea permisiunile necesare a dus la erori, motiv pentru care am adăugat o verificare în acest sens.
c) Am întâmpinat dificultăți în recunoașterea corectă a argumentelor, indifierent de ordinea în care acestea apar (dacă – follow-symlink este după director sau nu există)

4. Experimente
   a) Experiment 1
   b) Experiment 2 (cu temporary files)

a)

```
cleo@Ubuntu:~$ mkdir -p test/a test/b test/c
cleo@Ubuntu:~$ touch test/file1 test/file2
cleo@Ubuntu:~$ ln -s ../file1 test/a/link_to_file1
cleo@Ubuntu:~$ ln -s missing_file test/b/broken_link
cleo@Ubuntu:~$ ln -s ../b test/c/link_to_b
cleo@Ubuntu:~$ ln -s ../c test/b/link_to_c
cleo@Ubuntu:~$ mkdir test/protected
cleo@Ubuntu:~$ chmod 000 test/protected
cleo@Ubuntu:~$ ./slcheck.sh test
Analizez directorul: test
[BROKEN LINK] test/b/broken_link
Avertisment: Nu pot accesa directorul test/protected
cleo@Ubuntu:~$ ./slcheck.sh --follow-symlinks test
Analizez directorul: test
[BROKEN LINK] test/b/broken_link
[BROKEN LINK] test/b/link_to_c/link_to_b/broken_link
 Buclă detectată (inode deja vizitat): test/b/link_to_c/link_to_b/link_to_c
 Buclă detectată (inode deja vizitat): test/c/link_to_b
Avertisment: Nu pot accesa directorul test/protected
cleo@Ubuntu:~$ ./slcheck.sh
Utilizare: ./slcheck.sh [--follow-symlinks] <director>
cleo@Ubuntu:~$ ./slcheck.sh testX
Eroare: testX nu este un director valid
cleo@Ubuntu:~$ tree -L 2 test
test
├── a
│   └── link_to_file1 -> ../file1
├── b
│   ├── broken_link -> missing_file
│   └── link_to_c -> ../c
├── c
│   └── link_to_b -> ../b
├── file1
├── file2
└── protected  [error opening dir]

7 directories, 4 files
```

b)

```
ania@pricania:~/Documents/SLCheck$ cd ~/Documents/SLCheck
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk
ania@pricania:~/Documents/SLCheck$ sudo mount -t tmpfs -o size=20M tmpfs /tmp/ramdisk
[sudo] password for ania:
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/a
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/b
ania@pricania:~/Documents/SLCheck$ mkdir -p /tmp/ramdisk/test/c
ania@pricania:~/Documents/SLCheck$ touch /tmp/ramdisk/test/file1
ania@pricania:~/Documents/SLCheck$ touch /tmp/ramdisk/test/file2
ania@pricania:~/Documents/SLCheck$ ln -s missing_file /tmp/ramdisk/test/b/broken_link
ania@pricania:~/Documents/SLCheck$ ln -s ../file1 /tmp/ramdisk/test/a/link_to_file1
ania@pricania:~/Documents/SLCheck$ ln -s ../b /tmp/ramdisk/test/c/link_to_b
ania@pricania:~/Documents/SLCheck$ ln -s ../c /tmp/ramdisk/test/b/link_to_c
ania@pricania:~/Documents/SLCheck$ mkdir /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ sudo chmod 000 /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh /tmp/ramdisk/test
Analizez directorul: /tmp/ramdisk/test
[BROKEN LINK] /tmp/ramdisk/test/b/broken_link
Avertisment: Nu pot accesa directorul /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh --follow-symlinks /tmp/ramdisk/test
Analizez directorul: /tmp/ramdisk/test
[BROKEN LINK] /tmp/ramdisk/test/b/broken_link
[BROKEN LINK] /tmp/ramdisk/test/b/link_to_c/link_to_b/broken_link
[INFO] Ciclu detectat sau director deja vizitat: /tmp/ramdisk/test/b/link_to_c/link_to_b/link_to_c (Sarit)
[INFO] Ciclu detectat sau director deja vizitat: /tmp/ramdisk/test/c/link_to_b (Sarit)
Avertisment: Nu pot accesa directorul /tmp/ramdisk/test/protected
ania@pricania:~/Documents/SLCheck$ ln -s /tmp/ramdisk/test link_catre_ram_complex
ania@pricania:~/Documents/SLCheck$ sudo umount /tmp/ramdisk
ania@pricania:~/Documents/SLCheck$ ./SLCheck.sh link_catre_ram_complex
Eroare: link_catre_ram_complex nu este un director valid
```

5. Bibliografie

https://www.freecodecamp.org/news/symlink-tutorial-in-linux-how-to-create-and-remove-a-symbolic-link/
https://en.wikipedia.org/wiki/Symbolic_link
https://phoenixnap.com/kb/bash-associative-array
https://docs.rackspace.com/docs/what-are-inodes-in-linux